E-COMMERCE DATABASE
ERD & SCHEMA DOCUMENTATION

Formally Structured Entity-Relationship Diagram

Normalized to Third Normal Form (3NF)

Submitted By:

Muhammad Saad & Wajeeha Gul

BCS Group A — 4th Semester

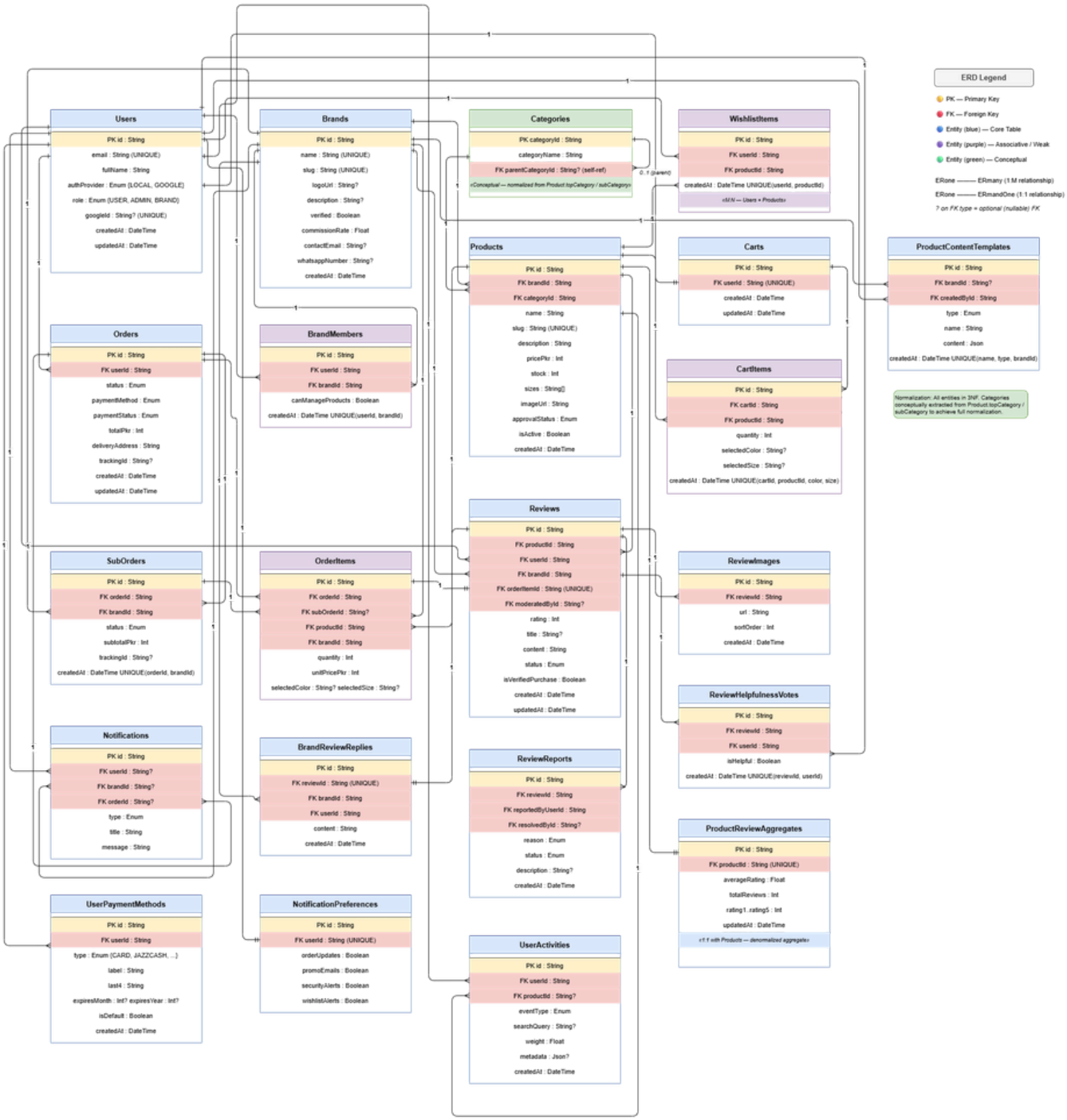
Submitted To:

Mr. Ali Hassan

Milestone	Date	Version	Remarks
ERD Diagram & Schema	4/24/2026	1.0	

ERD Diagram — Academic Entity-Relationship Diagram

The diagram below presents the complete normalized Entity-Relationship Diagram (ERD) for the E-Commerce marketplace database, modeled in accordance with academic data modeling standards. All entities are normalized to Third Normal Form (3NF). The **Categories** entity has been conceptually extracted from the flat ***topCategory/subCategory*** string fields in Products to achieve proper normalization.



Drawio:

https://app.diagrams.net/?src=about#G1vNG2oHPiCwn8_7oh7l6cojPHzJdJzp_U#%7B%22pageId%22%3A%22academic-erd%22%7D

Figure 1: Academic ERD — E-Commerce Database (19 Entities, 3NF Normalized)

Yellow row	Primary Key (PK) attribute
Red row	Foreign Key (FK) attribute
Blue entity	Core/strong entity table
Purple entity	Associative (junction) / weak entity
Green entity	Conceptual entity (normalized from denormalized fields)
1:M arrow	One-to-Many relationship (crow's foot notation)
1:1 arrow	One-to-One relationship (mandatory participation)

Data Dictionary — Entity Descriptions

The following section provides a complete data dictionary for each entity in the E-Commerce database, including attribute types, key roles, constraints, and normalization notes. The schema has been designed to support marketplace operations with multi-brand selling, order management, review systems, and user activity tracking.

1. Users

Thecentral strong entity representing registered users of the marketplace. A user can be a customer (role=USER), administrator (role=ADMIN), or brand owner (role=BRAND). Authentication supports both local credentials and Google OAuth. The entity participates in 1:1 relationships with ***Carts*** and ***NotificationPreferences***, and 1:M with ***Orders***, ***Reviews***, ***WishlistItems***, ***UserActivities***, and ***UserPaymentMethods***.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique user identifier (UUID)*
email	String	UNIQUE	*Login identifier — NOT NULL*
fullName	String	—	*Display name — NOT NULL*
authProvider	Enum	—	*LOCAL | GOOGLE*
role	Enum	—	*USER | ADMIN | BRAND*
googleId	String?	UNIQUE	*OAuth identifier — nullable*
createdAt	DateTime	—	*Row creation timestamp*
updatedAt	DateTime	—	*Last update timestamp*

2. Brands

Represents seller brands registered on the marketplace. Each brand has a unique slug for URL routing, a commission rate for revenue sharing, and optional contact information. Brands participate in 1:M with ***Products***, ***SubOrders***, ***OrderItems***, ***Reviews***, and via the ***BrandMembers*** associative table for M:N with ***Users***.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique brand identifier*
name	String	UNIQUE	*Brand display name — NOT NULL*
slug	String	UNIQUE	*URL-safe identifier — NOT NULL*
verified	Boolean	—	*Admin verification flag*
commissionRate	Float	—	*Marketplace revenue share %*
logoUrl	String?	—	*Brand logo image URL*
description	String?	—	*Brand bio text*
contactEmail	String?	—	*Public contact email*
whatsappNumber	String?	—	*WhatsApp contact number*
createdAt	DateTime	—	*Row creation timestamp*

3. Categories (Conceptual — Normalized Entity)

Normalizationnote: In theoriginal Prisma schema, categoryinformation was stored directly as string fields ***topCategory*** and ***subCategory*** inside the Products table — a violation of 2NF/3NF due to transitive dependency and update anomalies. For academic normalization, **Categories** is extracted as a proper entity with a self-referencing ***parentCategoryId*** to model the hierarchy (top-level → subcategory).

Attribute	Type	Key / Constraint	Notes
categoryId	String	PK	*Unique category identifier*
categoryName	String	—	*Category display name — NOT NULL*
parentCategoryId	String?	FK→self	*Null for top-level; points to parent*

4. Products

Coreproduct catalogue entity. Each product belongs to one brand and one category. Products carry a price snapshot in PKR, available sizes, a primary image URL, and an approval workflow status. Products participate in 1:M with ***CartItems***, ***WishlistItems***, ***OrderItems***, ***Reviews***, and 1:1 with ***ProductReviewAggregates***.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique product identifier*
brandId	String	FK → Brands	*Owner brand — NOT NULL*
categoryId	String	FK → Categories	*Product category — NOT NULL*
name	String	—	*Product name — NOT NULL*
slug	String	UNIQUE	*URL slug — NOT NULL*
pricePkr	Int	—	*Price in Pakistani Rupees*
stock	Int	—	*Available inventory count*
sizes	String[]	—	*Available size variants*
imageUrl	String	—	*Primary product image URL*
approvalStatus	Enum	—	*PENDING | APPROVED | REJECTED*
isActive	Boolean	—	*Listing visibility flag*
createdAt	DateTime	—	*Row creation timestamp*

5. Orders

Customer-level transaction records. An order captures the full purchase context: delivery address, payment method, payment status, and overall order status. The system uses a **split-order pattern** where each order is partitioned into ***SubOrders*** — one per brand — enabling independent fulfillment tracking per seller.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique order identifier*
userId	String	FK → Users	*Placing customer — NOT NULL*
status	Enum	—	*Order lifecycle status*
paymentMethod	Enum	—	*COD | CARD | JAZZCASH | etc.*

paymentStatus	Enum	—	*PENDING | PAID | FAILED*
totalPkr	Int	—	*Grand total in PKR*
deliveryAddress	String	—	*Shipping destination*
trackingId	String?	—	*Master tracking number*
createdAt	DateTime	—	*Order placement timestamp*

6. SubOrders

Brand-level fulfillment partitions of a parent order. Each SubOrder belongs to exactly one Order and one Brand, with its own status, subtotal, and tracking ID. The composite unique constraint ***(orderId, brandId)*** ensures at most one sub-order per brand per order.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique sub-order identifier*
orderId	String	FK → Orders	*Parent order — NOT NULL*
brandId	String	FK → Brands	*Fulfilling brand — NOT NULL*
status	Enum	—	*Fulfillment status*
subtotalPkr	Int	—	*Brand-level subtotal in PKR*
trackingId	String?	—	*Brand shipment tracking number*
createdAt	DateTime	UNIQUE(orderId, brandId)	*Creation timestamp*

7. OrderItems

OrderItems isthe major associative entity that resolves the M:N relationship between ***Orders*** and ***Products***. It also binds brand and sub-order fulfillment context, and records a price snapshot (***unitPricePkr***) at the time of purchase to maintain historical accuracy independent of future product price changes.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique order item identifier*
orderId	String	FK → Orders	*Parent order — NOT NULL*
subOrderId	String?	FK → SubOrders	*Brand sub-order (nullable)*
productId	String	FK → Products	*Purchased product — NOT NULL*
brandId	String	FK → Brands	*Product brand — NOT NULL*
quantity	Int	—	*Number of units ordered*
unitPricePkr	Int	—	*Price snapshot at purchase time*
selectedColor	String?	—	*Chosen color variant*
selectedSize	String?	—	*Chosen size variant*

8. Carts & 9. CartItems

Cartsmaintains a1:1relationship with ***Users*** — every user has exactly one active cart. ***CartItems*** is the junction entity between ***Carts*** and ***Products***, with a composite unique constraint on ***(cartId, productId, selectedColor, selectedSize)*** to prevent duplicate line entries.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique cart item identifier*
cartId	String	FK → Carts	*Parent cart — NOT NULL*
productId	String	FK → Products	*Added product — NOT NULL*
quantity	Int	—	*Quantity in cart*
selectedColor	String?	—	*Selected color variant*
selectedSize	String?	—	*Selected size variant*

10. WishlistItems

Resolves theM:N relationship between ***Users*** and ***Products*** for wishlisting. A composite unique constraint ***(userId, productId)*** prevents duplicate wishlist entries.

Attribute	Type	Key / Constraint	Notes
id	String	PK	Unique wishlist item identifier
userId	String	FK → Users	*Owner user — NOT NULL*
productId	String	FK → Products	*Wishlisted product — NOT NULL*
createdAt	DateTime	UNIQUE(userId, productId)	*Creation timestamp*

Review & Rating Subsystem

11. Reviews

The ***Reviews*** entity enforces **verified purchase linkage** via a unique FK to ***OrderItems.orderItemId*** — meaning each purchased line item can generate at most one review. This prevents fake reviews. The ***moderatedById*** FK to ***Users*** supports admin moderation workflows.

Attribute	Type	Key / Constraint	Notes
id	String	PK	*Unique review identifier*
productId	String	FK → Products	*Reviewed product*
userId	String	FK → Users	*Reviewer user*
brandId	String	FK → Brands	*Product brand*
orderItemId	String	FK → OrderItems (UNIQUE)	*Verified purchase link*
moderatedById	String?	FK → Users	*Moderating admin (nullable)*
rating	Int	—	*1–5 star rating*
title	String?	—	*Optional review headline*
content	String	—	*Review body text*
status	Enum	—	*PENDING | APPROVED | REJECTED*
isVerifiedPurchase	Boolean	—	*Purchase verification flag*

12–15. Review Sub-Entities

ReviewImages	Stores uploaded images for a review (1:M with Reviews). Attributes: id (PK), reviewId (FK), url, sortOrder, createdAt.
ReviewHelpfulnessVotes	M:N junction between Reviews and Users for helpfulness voting. Unique constraint (reviewId, userId). Attributes: id (PK), reviewId (FK), userId (FK), isHelpful (Boolean).
ReviewReports	Allows users to report inappropriate reviews (1:M with Reviews). Supports resolution workflow via resolvedById (FK → Users, nullable). Attributes: id (PK), reviewId (FK), reportedByUserId (FK), reason (Enum), status (Enum).
BrandReviewReplies	1:1 with Reviews — a brand can post exactly one official reply per review. Attributes: id (PK), reviewId (FK, UNIQUE), brandId (FK), userId (FK), content.
ProductReviewAggregates	1:1 denormalized aggregate with Products. Pre-computed averageRating, totalReviews, and individual star-count breakdowns (rating1–rating5) for query performance.

16–19. User Support Entities

BrandMembers	Resolves M:N between Users and Brands. Tracks staff membership with granular permissions (canManageProducts). Unique constraint (userId, brandId). Supports multi-staff brand management.
UserPaymentMethods	1:M with Users. Stores tokenized payment profiles (CARD, JAZZCASH, etc.) with masked card details. isDefault flag designates the preferred payment method.
NotificationPreferences	1:1 with Users. Per-user toggles for orderUpdates, promoEmails, securityAlerts, and wishlistAlerts notification channels.
Notifications	Broadcast-style notification records. Optional FKs to userId, brandId, and orderId allow targeted delivery. Supports system-wide or entity-specific notifications.
UserActivities	1:M with Users (and optional 1:M with Products). Records behavioral events (views, searches, purchases) with a weight float for recommendation scoring and metadata JSON for context.
ProductContentTemplates	Brand-scoped or global templates for product descriptions. Unique constraint (name, type, brandId). Enables reusable content patterns across product listings.

Relationship Map — Cardinalities & Constraints

One-to-Many (1:M) Relationships

Parent Entity	Child Entity	Cardinality	Foreign Key / Notes
Users	Orders	**1:M**	*Orders.userId → Users.id*
Users	Carts	**1:1**	*Carts.userId → Users.id (UNIQUE)*
Users	BrandMembers	**1:M**	*BrandMembers.userId → Users.id*
Users	Reviews	**1:M**	*Reviews.userId → Users.id*
Users	WishlistItems	**1:M**	*WishlistItems.userId → Users.id*
Users	NotificationPreferences	**1:1**	*NotificationPreferences.userId (UNIQUE)*
Users	UserPaymentMethods	**1:M**	*UserPaymentMethods.userId → Users.id*
Users	UserActivities	**1:M**	*UserActivities.userId → Users.id*
Users	Notifications	**1:M**	*Notifications.userId → Users.id (nullable)*
Brands	Products	**1:M**	*Products.brandId → Brands.id*
Brands	SubOrders	**1:M**	*SubOrders.brandId → Brands.id*
Brands	OrderItems	**1:M**	*OrderItems.brandId → Brands.id*
Brands	Reviews	**1:M**	*Reviews.brandId → Brands.id*
Brands	BrandMembers	**1:M**	*BrandMembers.brandId → Brands.id*
Brands	Notifications	**1:M**	*Notifications.brandId → Brands.id (nullable)*
Categories	Products	**1:M**	*Products.categoryId → Categories.categoryId*
Categories	Categories (sub)	**1:M**	*Self-ref: parentCategoryId → categoryId*
Orders	SubOrders	**1:M**	*SubOrders.orderId → Orders.id*
Orders	OrderItems	**1:M**	*OrderItems.orderId → Orders.id*
Orders	Notifications	**1:M**	*Notifications.orderId → Orders.id (nullable)*
SubOrders	OrderItems	**1:M**	*OrderItems.subOrderId → SubOrders.id (nullable)*
Products	CartItems	**1:M**	*CartItems.productId → Products.id*
Products	WishlistItems	**1:M**	*WishlistItems.productId → Products.id*
Products	OrderItems	**1:M**	*OrderItems.productId → Products.id*
Products	Reviews	**1:M**	*Reviews.productId → Products.id*
Products	ProductReviewAggregates	**1:1**	*ProductReviewAggregates.productId (UNIQUE)*
Products	UserActivities	**1:M**	*UserActivities.productId → Products.id (nullable)*
Carts	CartItems	**1:M**	*CartItems.cartId → Carts.id*
Reviews	ReviewImages	**1:M**	*ReviewImages.reviewId → Reviews.id*

Reviews	ReviewHelpfulnessVotes	**1:M**	*ReviewHelpfulnessVotes.reviewId → Reviews.id*
Reviews	ReviewReports	**1:M**	*ReviewReports.reviewId → Reviews.id*
Reviews	BrandReviewReplies	**1:1**	*BrandReviewReplies.reviewId (UNIQUE)*
OrderItems	Reviews	**1:1**	*Reviews.orderItemId → OrderItems.id (UNIQUE)*

Many-to-Many (M:N) Relationships — Resolved via Junction Tables

Parent Entity	Child Entity	Cardinality	Foreign Key / Notes
Users	Brands	**M:N → BrandMembers**	*Staff membership with permissions*
Orders	Products	**M:N → OrderItems**	*Core transactional junction*
Users	Products	**M:N → WishlistItems**	*User product wishlist*
Carts	Products	**M:N → CartItems**	*Shopping cart contents (via Carts)*

Database Schema — Summary Table

The following table provides a high-level schema summary for all 19 entities, organized by functional domain. Colors denote entity classification.

Core Users & Identity

Entity	Attributes	Size
Users	*id, email, fullName, authProvider, role, googleId, createdAt, updatedAt*	8 cols
BrandMembers	*id, userId(FK), brandId(FK), canManageProducts, createdAt*	5 cols · M:N junction

Brands & Catalogue

Entity	Attributes	Size
Brands	*id, name, slug, logoUrl, description, verified, commissionRate, contactEmail, whatsappNumber, createdAt*	10 cols
Categories	*categoryId, categoryName, parentCategoryId(FK→self)*	3 cols · conceptual
Products	*id, brandId(FK), categoryId(FK), name, slug, pricePkr, stock, sizes, imageUrl, approvalStatus, isActive, createdAt*	12 cols
ProductContentTemplates	*id, brandId(FK?), createdById(FK), type, name, content, createdAt*	7 cols

Shopping

Entity	Attributes	Size
Carts	*id, userId(FK), createdAt, updatedAt*	4 cols
CartItems	*id, cartId(FK), productId(FK), quantity, selectedColor, selectedSize, createdAt*	7 cols · junction
WishlistItems	*id, userId(FK), productId(FK), createdAt*	4 cols · junction

Orders & Fulfilment

Entity	Attributes	Size
Orders	*id, userId(FK), status, paymentMethod, paymentStatus, totalPkr, deliveryAddress, trackingId, createdAt*	9 cols
SubOrders	*id, orderId(FK), brandId(FK), status, subtotalPkr, trackingId, createdAt*	7 cols
OrderItems	*id, orderId(FK), subOrderId(FK?), productId(FK), brandId(FK), quantity, unitPricePkr, selectedColor, selectedSize*	9 cols · junction

Reviews & Ratings

Entity	Attributes	Size
Reviews	*id, productId(FK), userId(FK), brandId(FK), orderItemId(FK unique), moderatedById(FK?), rating, title, content, status, isVerifiedPurchase, createdAt*	12 cols
ReviewImages	*id, reviewId(FK), url, sortOrder, createdAt*	5 cols
ReviewHelpfulnessVotes	*id, reviewId(FK), userId(FK), isHelpful, createdAt*	5 cols · junction
ReviewReports	*id, reviewId(FK), reportedByUserId(FK), resolvedById(FK?), reason, status, description, createdAt*	8 cols
BrandReviewReplies	*id, reviewId(FK unique), brandId(FK), userId(FK), content, createdAt*	6 cols
ProductReviewAggregates	*id, productId(FK unique), averageRating, totalReviews, rating1–5, updatedAt*	9 cols

Notifications & Activity

Entity	Attributes	Size

Notifications	*id, userId(FK?), brandId(FK?), orderId(FK?), type, title, message, readAt, createdAt*	9 cols
NotificationPreferences	*id, userId(FK unique), orderUpdates, promoEmails, securityAlerts, wishlistAlerts*	6 cols
UserPaymentMethods	*id, userId(FK), type, label, last4, expiresMonth, expiresYear, isDefault, createdAt*	9 cols
UserActivities	*id, userId(FK), productId(FK?), eventType, searchQuery, weight, metadata, createdAt*	8 cols

Normalization Analysis & Design Notes

First Normal Form (1NF)

Allattributes hold atomicvalues. The ***sizes*** array field in Products is a known implementation concession (stored as String[] in PostgreSQL); for strict 1NF compliance, a **ProductSizes** weak entity would be used. All tables have a defined primary key.

Second Normal Form (2NF)

Allnon-key attributes are fully functionally dependent on the entire primary key. The most significant 2NF correction is the extraction of **Categories** from the Products table, where topCategory/subCategory were partially dependent on a conceptual category identifier rather than the product's PK.

Third Normal Form (3NF)

Notransitive dependencies exist. **ProductReviewAggregates** is a deliberate denormalization for query performance (pre-computed averages), clearly documented as such. All other non-key attributes depend directly and only on their entity's PK.

Referential Integrity

Allforeignkeys are properly declared. Nullable FKs (marked ?) model optional participation. The ***ON DELETE SET NULL*** pattern would apply to volunteer/user FKs where records should be preserved on deletion. Cascade deletes apply to strong-weak entity pairs (e.g., Reviews → ReviewImages).

M:N Resolution

Allmany-to-manyrelationships are resolved through explicit associative entities: BrandMembers (Users × Brands), OrderItems (Orders × Products), CartItems (Carts × Products), WishlistItems (Users × Products), ReviewHelpfulnessVotes (Reviews × Users).

Total Entities: 19 | Core Strong Entities: 9 | Associative Entities: 5 | Support Entities: 4 | Conceptual Entities: 1